

DriftLab: Measuring Agent Adaptation in Non-Stationary Environments

Nassim Berrada

Unfolding Labs

nassim@unfoldinglabs.ai

Preprint. Code: github.com/unfoldinglabs/driftlab

Abstract

Benchmarks for language-model agents evaluate competence on frozen worlds: the task, its rules, and its ground truth are fixed before the first step. Conversely, deployed agents work in environments whose rules change constantly, sometimes with notice, usually without, and sometimes because of the agent’s own behavior. We present **DriftLab**, a testbed for studying the dynamics of AI agent adaptation in non-stationary environments. Each defined environment is a deterministic, seeded simulation where the agent never sees the ground truth. Adaptation therefore becomes measurable as a time course: how many encounters with an affected task an agent needs to notice a change, to recover, and to make the new rule stick. Drift itself is the controlled variable, spanning abrupt, gradual, cosmetic, endogenous, and adversarial change, where an opponent aims drift at the agent’s competence. Agents are evaluated through one loop whose memory substrate is swappable behind a fixed model, so memory itself becomes the independent variable. By treating latent world changes as a first-class testable parameter, DriftLab provides a way to measure the intelligence of agentic systems in the context of turbulent environments.

1 Introduction

The benchmarks that shaped the current generation of language-model agents share a tacit assumption: the environment does not change over the course of the evaluation. SWE-bench freezes a repository at a commit (Jimenez et al., 2024). GAIA fixes its questions and answers (Mialon et al., 2023). WebArena resets its websites between episodes (Zhou et al., 2023). AgentBench evaluates across eight environments, all stationary within an episode (Liu et al., 2024). τ -bench varies the simulated user, never the rules of the domain (Yao et al., 2024). Under this assumption, competence is a scalar: the fraction of tasks solved.

In deployment, this assumption rarely holds. A routing policy an operations agent learned last month is quietly revised. A schema gains a field and tightens a format. Demand shifts under an inventory controller. A style guide changes between a developer agent’s tickets. In adversarial settings such as fraud, spam, and security, the environment is not merely non-stationary: it shifts in response to the agent’s own behavior. In all of these cases the interesting quantity is the *dynamics* of competence: how quickly the agent notices that something changed, how quickly it recovers, whether the new knowledge sticks, whether learning it damaged anything else, and whether the agent can tell a changed world from a noisy one.

Measuring dynamics requires ground truth about the change itself. This is necessary to know when the change happened, which tasks it affected, and what the correct behavior is at every step, before and after. DriftLab is built around this requirement. Every environment is a simulation that knows its own hidden state. At every step, the evaluation loop writes that state next to the exchange into a run record the agent never sees and nothing ever edits. All metrics are computed from the record after the fact. Because the truth is recorded rather than inferred, adaptation can be counted in its natural unit, encounters with affected tasks, and a metric defined later applies retroactively to every run ever recorded.

This paper makes three contributions:

1. **Environments with hidden, recorded ground truth** (Section 3.1, Appendix A): six workplace simulations in which the agent is never told it is being evaluated and the world’s true state is logged at every step.
2. **Drift as a controlled variable** (Section 3.2): a taxonomy spanning abrupt and gradual substantive change, cosmetic change (only appearances move,

so reacting is the error, endogenous change (the agent’s own behavior triggers it), and adversarial change (a co-adapting opponent aims drift at the agent’s competence). Gradual drift is constructed so ground truth stays deterministic at every step.

3. **Adaptation metrics** (Section 3.3): encounter-based detection, recovery, and stability latencies; retention; stale-memory rate; interference; calibration and confidence lead; and an over-update rate that measures abandoning still-correct behavior under misleading feedback. A capability score aggregates adaptation, knowledge, and epistemics; cost is reported beside it.

Section 4 describes the evaluation protocol, Section 5 reports benchmark characterization and registered studies as demonstrations of the instrument, and Section 6 states limitations and future work. Full specifications of the worlds, the twenty experimental protocols, and the memory substrates are in Appendices A through C.

2 Related work

Agent benchmarks. SWE-bench (Jimenez et al., 2024), GAIA (Mialon et al., 2023), WebArena (Zhou et al., 2023), AgentBench (Liu et al., 2024), and τ -bench (Yao et al., 2024) evaluate task competence in environments that are fixed within, and usually across, episodes. τ -bench’s pass^k metric probes reliability under stochastic replay, which is orthogonal to non-stationarity: the world is identical on every trial. DriftLab studies the environment’s change process rather than the task.

Benchmarks with a time axis. StreamBench (Wu et al., 2024) asks whether agents improve over an input-feedback sequence, with a stationary task distribution underneath: it measures improvement, not adaptation to change. LongMemEval (Wu et al., 2025) tests long-term conversational memory, including knowledge updates, but the assistant’s world changes only through what interlocutors say. Closest to our setting, the contemporaneous EvoArena (Xu et al., 2026) evaluates agents under progressive environment updates to terminals, repositories, and user preferences, and proposes a patch-based memory (EvoMem). DriftLab differs on three axes. First, recorded ground truth makes dynamics measurable: detection and recovery latency in affected-task encounters, retention, and

staleness, where update benchmarks measure accuracy after updates. Second, the drift taxonomy includes cosmetic, endogenous, and adversarial change, which accuracy-under-updates settings do not distinguish. Third, the statistical protocol is part of the artifact.

Continual and non-stationary learning. Catastrophic interference (McCloskey and Cohen, 1989; Kirkpatrick et al., 2017) and its measurement through backward transfer (Lopez-Paz and Ranzato, 2017) motivate our interference metric, transplanted from weight-space learning to in-context and memory-mediated learning. Continual RL formalizes learning under non-stationary processes (Khetarpal et al., 2022), and recent model-based work shows that limiting the influence of stale data is necessary for calibrated uncertainty under drift (Iten et al., 2026), the intuition behind our staleness and retention metrics. The concept-drift literature (Gama et al., 2014; Cabrera Martin et al., 2025) supplies the vocabulary of abrupt versus gradual drift; non-stationary bandits (Garivier and Moulines, 2011) supply regret analyses under switching rewards. DriftLab operationalizes these constructs for agents whose policy is a prompt-conditioned generation and whose weights, in effect, are a memory artifact.

Agent memory. Reflexion (Shinn et al., 2023) converts feedback into verbal self-reflections; Voyager (Wang et al., 2023) grows a skill library; Generative Agents (Park et al., 2023) synthesize memories through reflection; MemGPT (Packer et al., 2023) manages memory tiers. These works propose mechanisms; DriftLab provides an instrument for comparing them under change, with the memory substrate as the independent variable, including a world-model substrate with explicit validity conditions that revises itself when its own predictions fail, in the spirit of the evolver level of agentic world modeling (Chu et al., 2026).

Calibration and volatility. Verbalized confidence is often miscalibrated (Xiong et al., 2024; Tian et al., 2023). DriftLab asks a sharper temporal question: does confidence sag on affected tasks before behavior recovers, making it a usable change detector? Literature shows that people adjust their learning rates to environmental volatility (Behrens et al., 2007) and distinguish change-points from noise (Nassar et al., 2010); our over-update metric tests whether language agents make the same distinction.

3 The DriftLab design

3.1 Setting and notation

Definition 1 (World and episode). An episode is a sequence of T exchanges between an agent and a world. At step t the world renders an observation x_t from its hidden state, the agent replies with text, the world parses the reply into an action a_t and returns a reward r_t and in-role feedback. In worlds with discrete tasks, each task carries a key κ_t identifying its kind, and the hidden state determines a truth map f_t from keys to correct actions, so the correct action at step t is $f_t(\kappa_t)$. The agent never observes f_t , the key, or any of the machinery around it. Six worlds instantiate this contract, workplace simulations specified in Appendix A: routing emailed requests, entering orders from prose intake notes, controlling inventory under retail-shaped demand, building code on an evolving internal library, screening insurance claims against an adaptive fraud ring, and choosing campaign angles under pooled-only feedback.

Definition 2 (Drift and change events). **Drift** is a change to f_t (or, in continuous worlds, to the reward process). A change event $c = (t_c, A_c)$ occurs at step t_c and affects a key set A_c .

Definition 3 (Encounter). For a key $\kappa \in A_c$, the k -th **encounter** after change c is the k -th step $t > t_c$ with $\kappa_t = \kappa$. Counting adaptation in encounters rather than steps removes the sampling frequency of the environment from the measurement of the agent: a change the agent met twice is not scored like one it met forty times.

Every world is **seeded**: a single integer seed determines all of its randomness, from the task sequence and the drift schedule down to surface detail such as names and narrative text. An episode is therefore a deterministic function of its seed, and two agents given the same seed face byte-identical worlds. This is what makes the paired statistics of Section 4.4 possible. The surface detail carries no signal about hidden state.

The evaluation loop writes one entry per step into the **run record** (Figure 1): the observation, the agent’s reply and parsed action, the reward and feedback, the key, the correct action $f_t(\kappa_t)$, any change events applied at t , and the agent’s stated confidence when one was elicited. One boundary completes the design: the agent

never reads world state, and the framework never reads agent memory; the two sides meet only as text.

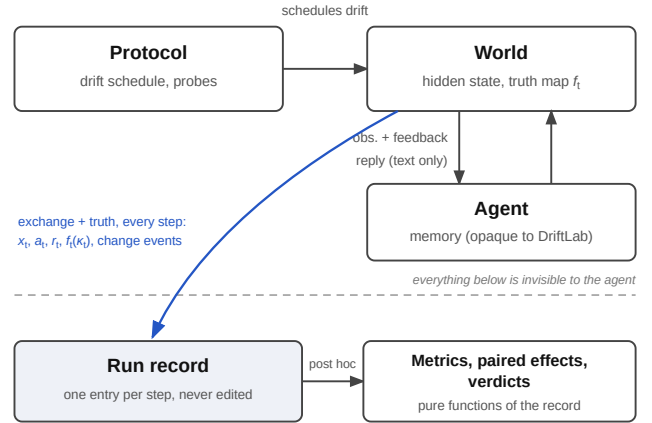


Figure 1: The DriftLab measurement loop. The protocol schedules drift in a seeded world; the agent and the world exchange only text; and the evaluation loop writes each exchange, together with the hidden truth, into an immutable run record from which all metrics, paired effects, and verdicts are computed after the fact.

3.2 Drift as the controlled variable

DriftLab separates what changes (owned by the world) from when it changes (owned by the experimental protocol). Five topologies are exercised.

Abrupt substantive drift. f_t changes at a scheduled step: a rule flips, a schema mutates, a demand rate jumps. This is the classic change-point setting (Gama et al., 2014).

Gradual substantive drift, with deterministic truth. Real change often arrives as a transition rather than a discontinuity, but a naively randomized transition invalidates the measurement, since during the transition there is no longer a correct answer to score against. DriftLab’s gradual drift keeps f_t deterministic at every step. In the routing world, a flipped rule migrates its affected keys a few per step through temporary exceptions. In the fraud world, the adversary runs old and new playbooks simultaneously while its traffic mix shifts; both patterns are fraudulent during the handover, and the old one retires at the end.

Cosmetic drift. Only appearances move: the task is re-rendered in a different format with inverted field order, labels rotate, narrative templates change. f_t is untouched, so any behavioral reaction is over-reaction.

Endogenous drift. The world changes because of the agent: an overloaded desk sheds work, a supplier strains under a large order, a reviewer starts expecting the agent’s own habits, the IT department builds an alias around the agent’s persistent mistake. No notice marks the change; the only evidence of it is in the agent’s own interaction history.

Adversarial drift. In the fraud world, the ring observes outcomes: when too many of its claims are caught, it re-styles them to mimic the traffic the agent has recently approved. Drift is therefore concentrated where the agent’s current policy is most permissive, and the agent’s own success is what triggers the next shift.

Protocols are packaged as twenty versioned **experiments** (Appendix B). An experiment fixes a schedule of drift, notices, and probes over any world that supports it, and declares its **regimes**: the settings of its independent variable, such as scheduled versus adversarial drift, a feedback delay of zero versus six steps, or no drift versus twelve changes. Figure 2 shows the division of labor: the world owns what a change means and implements each mechanism as an inert hook or flag; the protocol owns when and under which conditions anything fires, and a regime is one named setting of that choice. Each protocol enables one change source at a time, so, for example, endogenous effects are studied against a scheduled control rather than as a confound.

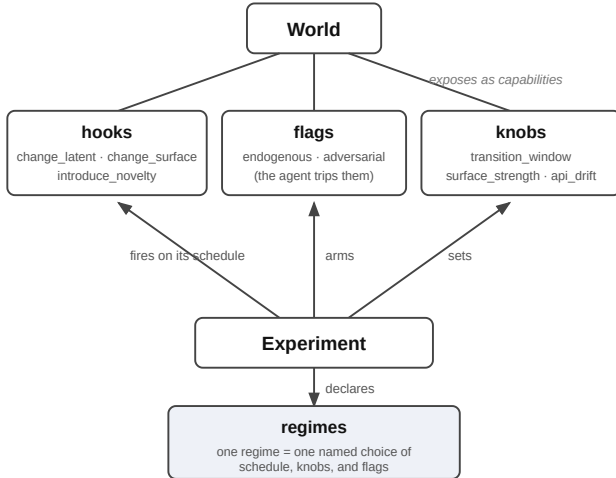


Figure 2: The division of labor behind drift, as entities and their connections. A world exposes change hooks (called to move the truth or its surface), flags (mechanisms the agent’s own behavior trips), and knobs (how a change unfolds), all inert on their own. An experiment fires the hooks on its

schedule, arms the flags, sets the knobs, and declares its regimes: each regime is one named choice of schedule, knobs, and flags.

3.3 Measurement

All metrics are pure functions of the run record, computed after the fact, so a metric defined later applies retroactively to every recorded run. For a change c with affected keys A_c , let $e_1, e_2, \dots$ be the post-change encounters with keys in A_c ; write a_k and κ_k for the action and key at encounter e_k , $y_k = \mathbf{1}[a_k = f(\kappa_k)]$ for correctness against the current truth, and f^- for the truth map just before t_c . Figure 3 illustrates the anatomy of one change.

Detection latency asks how many affected-task encounters elapse before the agent stops applying the obsolete rule. Formally, $d(c) = \min\{k : a_k \neq f^-(\kappa_k)\}$: encounters before the old answer stops appearing.

Recovery and stability latency distinguish first success from reliable adaptation. Recovery asks when the new answer first appears; stability asks when it begins to persist. Recovery $\rho(c) = \min\{k : y_k = 1\}$ counts encounters before the new correct answer first appears; stability $\sigma(c) = \min\{k : y_k = y_{k+1} = 1\}$ requires it to hold twice in a row.

Retention asks whether a learned change sticks rather than merely appearing once. It is $R(c) = \frac{1}{|W_c|} \sum_{k \in W_c} y_k$, where W_c spans the 9th through the 38th post-change encounter, censored at the next change to the same keys.

Stale-memory rate measures relapse: how often an agent returns to the obsolete answer after demonstrating that it can produce the new one. It is $S(c) = \frac{1}{|K_c|} \sum_{k \in K_c} \mathbf{1}[a_k = f^-(\kappa_k)]$, where K_c is the encounters after the agent’s first post-change success (after a five-encounter grace period when it never succeeds).

Interference asks whether updating affected tasks damages unrelated behavior. It is $I(c) = \bar{y}_{\text{after}}^{\text{unaff}} - \bar{y}_{\text{before}}^{\text{unaff}}$: accuracy on *unaffected* keys in 15-step windows after versus before the change; a negative value is collateral damage in the sense of backward transfer (Lopez-Paz and Ranzato, 2017).

Over-update rate asks whether an agent treats misleading negative feedback as evidence that a correct rule changed. It is $U = \Pr(a_{\text{next}(\kappa)} \neq a \mid a = f(\kappa), r \leq$

0): among events where feedback wrongly punished a correct action (possible only in worlds with misleading feedback), the share of next same-key encounters at which the agent abandoned the still-correct behavior. It requires at least one qualifying event.

Calibration and overconfidence measure whether expressed certainty tracks reality, rather than merely whether the final action was correct. With stated confidences p_i and outcomes $o_i \in \{0, 1\}$, calibration error is the Brier score $B = \frac{1}{n} \sum_i (p_i - o_i)^2$ and overconfidence is $\bar{p} - \bar{o}$; both require at least five stated confidences.

Confidence lag and lead ask whether the agent’s stated uncertainty notices a change before its actions do. The lag $\lambda(c)$ is the encounters until stated confidence on affected tasks falls below its pre-change baseline; the lead $\rho(c) - \lambda(c)$ is positive when confidence registered the change before behavior recovered.

Wrong-prediction rate measures the frequency with which explicit forecasts are contradicted by the world, over at least five stated predictions.

Parse-failure rate and cost per success describe operational reliability: whether an answer can be consumed by the environment and what it costs to obtain a successful answer. They feed the efficiency dimension and never a capability one.

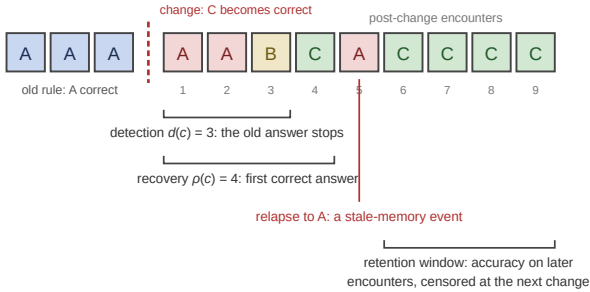


Figure 3: Anatomy of one change, schematically. Before the change, A is correct for this key. After it, the agent repeats A for two encounters (detected at the third, when the old answer stops), first produces the new correct answer C at the fourth (recovery), relapses once to the obsolete A (a stale-memory event), and then holds C through the retention window. Counting in encounters with affected keys, rather than steps, removes the environment’s sampling frequency from the measurement.

The raw measurements are scored against recorded truth, not reported reward, so a world that lies in its feedback or withholds it is still evaluated against reality. Section 3.4 defines how these measurements become dimensions and a summary score.

3.4 Scoring

The four reported dimensions answer different practical questions. **Adaptation** asks how quickly the agent notices and recovers from a changed rule. **Knowledge** asks how accurately it performs and whether the update is retained without damaging unaffected tasks. **Epistemics** asks whether its memory and confidence track what is actually true, including resistance to stale beliefs and misleading feedback. **Efficiency** asks whether the agent produces usable answers at reasonable operational cost. These dimensions are reported together; they should not be read as interchangeable measures of intelligence.

For each dimension, valid component metrics are first mapped to $[0, 1]$, with larger values always better. Let $q(\ell) = 1/(1 + \ell/5)$ be the latency transform, and let $\text{clip}_{[0,1]}$ bound a quantity to the unit interval. Adaptation is the mean of the available transformed detection and recovery latencies, with a component scored as zero when the agent had a fair chance but never adapted:

$$A = \text{mean}(q(d), q(\rho)).$$

Knowledge combines performance and resistance to collateral damage:

Let $I^* = \text{clip}_{[0,1]}(1 + 2 \min(0, I))$ be the normalized interference score. Then

$$K = \text{mean}(\text{final accuracy}, R, I^*).$$

Epistemics rewards calibrated confidence and resistance to stale or unjustified updating. The implementation uses the complements of error rates, so that higher is better:

$$E = \text{mean}(1 - B, 1 - S, 1 - U),$$

omitting components that the world or agent does not make measurable; confidence lead and wrong-prediction rate remain reported diagnostics rather than current score components. Efficiency combines parse compliance

with the cost transform. Let P be parse-failure rate and C be cost per success:

$$F = \text{mean}(1 - P, 1/(1 + C/\$0.01)).$$

The headline **DriftLab score** is the capability summary, not a replacement for the vector:

$$D = (A + K + E)/3.$$

Efficiency F , including dollars, is reported beside D and is never blended into it; a cheaper model must not receive a capability increase merely for being cheap. A metric that a world, agent, or regime cannot ask about is missing and excluded from the relevant mean, never silently converted to zero. Runs below the 30-step validity floor do not enter profiles. In worlds without repeating task keys, latency can fall back to time-based recovery of a rolling reward, while accuracy-style metrics remain unmeasured; continuous-cost worlds are scored on reward rather than invented accuracy. The constants in these transforms set a readable scale, not a scientific natural unit, so the dimension vector and its scope are the primary result and D is only a ranking convenience.

4 Evaluation protocol

4.1 Agents

The entire agent contract is `act(prompt) → text` and `observe(feedback, reward)`. A **profile** is a declarative record containing the agent type, pinned model, reasoning and output limits, memory-substrate configuration, and cost metadata. The same episode loop instantiates each profile, so changing a profile changes a named experimental factor rather than silently changing the harness.

Reference agents are in-process LLM loops whose memory substrate is the controlled variable. The model set covers gpt-5.6-luna and two open-weights models, deepseek-v4.1-flash and qwen3.8-27b, all served through one OpenAI-compatible API layer with pinned identifiers and a per-call price table. They were chosen for the best intelligence-to-cost profile available at the time rather than for peak capability. A non-LLM floor—an ϵ -greedy, recency-weighted lookup table—plays the discrete worlds through the same contract.

The agent profile fixes the model and substrate for an episode and exposes the same observation, action, feedback, and reward fields to every arm. Prompts, token caps, reasoning effort, and consolidation settings are recorded with the profile; [Appendix C](#) gives the exact profile families and memory prompts. This makes model comparisons interpretable as model comparisons and substrate comparisons interpretable as model-substrate comparisons.

4.2 Memory

The seven memory profiles span a spectrum from remembering what happened to representing what currently holds:

None persists nothing; every step is played from the observation alone. It is the floor that any memory must beat, and the one agent a rule change cannot wrong-foot.

Transcript keeps a sliding window of raw exchanges, verbatim and unprocessed. Detection is cheap, because contradicting evidence sits in plain view, but nothing older than the window exists and nothing is ever revised.

Notes maintains a free-form working file the model rewrites on an interval or after failures. It retains beyond any window, at the price of consolidation lag and lossy rewrites: between rewrites the agent acts on a summary that may already be stale.

Skills extracts one-line condition-action procedures. The format matches worlds whose truth is policy-shaped and constrains worlds whose truth is a quantity or a belief.

Beliefs stores declarative statements, each with a confidence and an explicit condition that would revise it. It is the lightest substrate that can retire knowledge deliberately rather than by overwriting.

World model stores scoped rule hypotheses with confidences, age stamps, and invalidation conditions, and is the only substrate that changes the agent’s behavior loop: the agent predicts every outcome, and a failed prediction triggers an immediate revision instead of waiting for the interval.

Fast+slow carries a short raw transcript and consolidated notes at once, trading tokens for coverage of both timescales.

The substrate is the only state carried between steps by the agent. A raw transcript has no invalidation mechanism, a free-form summary can be revised only by lossy rewrite, and only rules with validity conditions map “the world changed” to a discrete, inspectable edit. The non-LLM lookup floor has its own tabular substrate and is included only where the world exposes discrete keys.

4.3 Episodes

An episode is a seeded sequence of environment exchanges. The world emits an observation, the profile returns an action, and the world records truth, reward, feedback, latency, and any drift event before the next exchange. The truth is hidden from the agent but retained in the run record for scoring. Each cell is an agent profile $\times$ world $\times$ regime $\times$ seed; paired cells therefore receive the same seed and the same exogenous event schedule.

Experiments cross every registered regime with their seed set. Runs are resumable and written under an identity that includes experiment, world, regime, seed, model, and substrate, so a completed run is not silently replaced by a later alias or quick probe. Timeouts and unparseable outputs remain observable failures with the protocol’s documented fallback behavior. Episode length, model settings, substrate settings, and API cost are recorded alongside the trajectory; they are protocol metadata, not hidden tuning.

4.4 Validation

An experiment runs as a **grid**: every regime is crossed with a set of seeds, and every agent plays the identical resulting episodes. All comparisons are therefore paired.

Definition 4 (Paired effect). For a metric m and two conditions A , B (two agents, or two regimes within an agent) matched on seeds $s \in S$, the effect is

$$\Delta = \frac{1}{|S|} \sum_{s \in S} [m_B(s) - m_A(s)],$$

reported with a 95% bootstrap confidence interval over the matched differences, never as a difference of unpaired means.

Experiments **pre-register predictions**: before any data is collected, the experiment’s registry entry states

what its outcome should look like, in one of three forms software can check. A *directional effect* names a metric and two conditions (“recovery latency is higher under adversarial drift than under scheduled drift”). A *reversal* claims that a ranking of agents flips between two regimes (“the short-memory arm wins under high drift and loses under none”). A *sign claim* states that a metric’s mean falls on one side of zero (“confidence lead is positive after a change”). An automated validator evaluates each prediction against the paired effects: *supported* when the interval excludes zero in the predicted direction over at least three matched pairs, *contradicted* on the other side, *inconclusive* otherwise. Verdicts are always scope-qualified, naming the agents and worlds actually tested, and they aggregate into proposed hypothesis statuses that a human confirms; the machinery never edits the research log on its own.

The run ledger records seed count and episode length for every completed cell. The validator selects the longest completed protocol available for each model–substrate identity when an experiment has extensions, and never pairs cells from different episode protocols. This preserves the audit trail while allowing the results section to treat completed runs as representative of their configured protocol.

What the verdicts are. Automated verdicts are statistical proposals over registered predictions. They inherit the registry’s choices about what was worth predicting, and they are always scoped: “supported, for these agents, on these worlds” is the strongest statement the machinery can make.

5 Results

5.1 Characterization

The characterization matrix. Table 1 reports the score vector rather than one value per experiment. Each row is a complete model–world cell from the available notes-profile runs. These are characterization cells, not a claim that every model has identical coverage.

Table 1: Adaptation, knowledge, efficiency, and available capability score per model and world. Only rows with all displayed components measured are included. Values are on $[0, 1]$, higher is better.

World	Model	Adaptation	Knowledge	Efficiency	Score
rule	deepseek-v4.1-flash	0.706	0.921	0.904	0.832
rule	gpt-5.6-luna	0.693	0.898	0.978	0.820
rule	qwen3.8-27b	0.774	0.880	0.911	0.822
claims	deepseek-v4.1-flash	0.805	0.823	0.963	0.814
claims	gpt-5.6-luna	0.920	0.761	0.963	0.893
claims	qwen3.8-27b	0.772	0.881	0.919	0.826
campaign	deepseek-v4.1-flash	0.232	0.088	0.764	0.160
campaign	gpt-5.6-luna	0.214	0.202	0.899	0.208
campaign	qwen3.8-27b	0.217	0.181	0.700	0.199

Cells with an unmeasured component are omitted from this compact comparison, not zero-filled; the coverage caveat is discussed below.

The rule world gives the cleanest cross-model comparison. DeepSeek has the strongest knowledge value, Luna has the highest operational efficiency, and Qwen adapts fastest. Their available capability scores are close, so the practical conclusion is a trade-off rather than a universal winner.

The form and inventory cells are omitted because at least one displayed component is unmeasured there; this is a property of the measurement contract, not a failed experiment. In the claims world, Luna has the highest adaptation and available score, while the open models have stronger knowledge values. Finally, campaign is difficult for all three models: adaptation and knowledge collapse together while efficiency remains moderate.

5.2 Registered studies

Four studies demonstrate what the instrument can establish, refute, and catch itself overreading. Each runs one protocol under its registered predictions, with seed-paired effects and scope-qualified verdicts.

Study 1: over-reaction to purely cosmetic change. The protocol places three events in an episode. Depending on the regime, each event changes nothing, changes only the task’s appearance, changes its substance, or changes both. A cosmetic event leaves the correct behavior untouched, so any accuracy an agent loses after one is over-reaction by definition.

Result: over-reaction happens, and where it happens depends on the world. On the form world, the record’s format is the very thing the agent works on, and

restructuring it cost the transcript agent a measurable share of its accuracy even though the schema never moved. On the routing world, where the same re-rendering is incidental to the job, no arm reacted at all. Agents shrug off appearance changes where appearance is incidental and over-react to them where appearance is the working material.

Evidence: Figure 4 shows every paired effect, cosmetic-only minus no-change, in final accuracy. The transcript agent is at -0.133 on the form world (95% CI $[-0.240, -0.025]$; supported). Notes move in the same direction without separating from zero (-0.169 , $[-0.360, +0.040]$), and both routing-world arms sit near zero. The open-model estimates are directionally mixed and their intervals include zero.

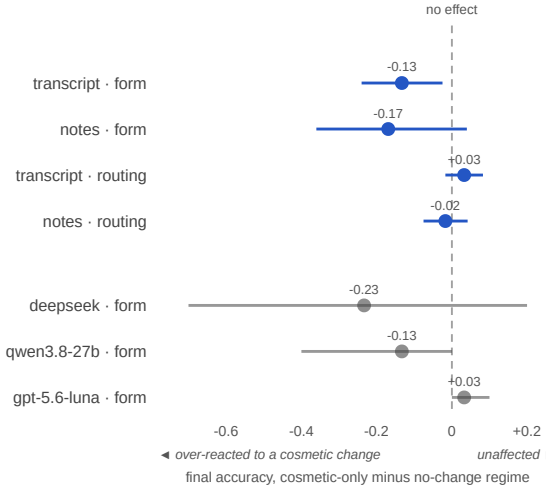


Figure 4: The cosmetic over-reaction effect across arms, worlds, and models. Each row is the paired difference in final accuracy between the cosmetic-only and no-change regimes; whiskers are 95% bootstrap intervals, and a point left of zero means the agent lost accuracy to a change that altered nothing but appearance. Rows identify model, memory substrate, and world; every completed row is treated as representative of that configured protocol.

Study 2: what adversarial drift costs. In the fraud world, the same number of tactic shifts can arrive on a timer or be aimed: the ring watches which of its claims get through and re-styles them against whatever the agent is currently approving. The registered prediction was that recovering from an aimed shift takes longer than recovering from a scheduled one.

Result: the available data do not establish a general per-shift recovery cost for aimed drift. Luna shows no positive latency effect, the DeepSeek estimates point slightly in the opposite direction, and the Qwen estimate leans positive but remains wide. The adversary does impose a different cost on Luna: accuracy stays lower for the whole episode, and the ring shifts more often the better the agent does, so the pressure never lets up.

Evidence: Figure 5 shows recovery-latency deltas, adversarial minus scheduled. Luna is -0.67 encounters $[-1.61, +0.13]$ with notes and -0.56 $[-2.14, +1.15]$ with a transcript; DeepSeek is -0.31 $[-1.00, +0.62]$ and -0.02 $[-0.40, +0.33]$, respectively. Qwen is $+0.944$ $[-1.000, +2.333]$, while an earlier DeepSeek estimate of $+1.278$ $[+0.333, +2.000]$ does not persist in the completed extension. The accuracy gap on Luna is clear (scheduled minus adversarial: $+0.100$ $[0.027, 0.173]$ for notes and $+0.247$ $[0.127, 0.360]$ for transcript). The

shift count is endogenous: against the notes agent the ring moved 34 times where the scheduled control moved 20.

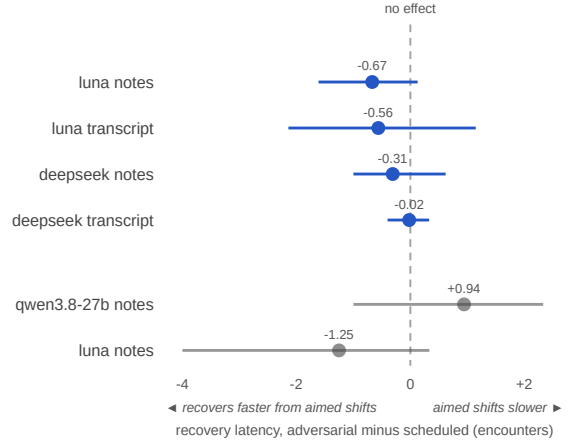


Figure 5: Recovery latency under aimed versus scheduled drift, adversarial minus scheduled, in encounters. A point right of zero means aimed shifts took longer to recover from. Rows identify model and memory substrate; the estimates are interpreted within their configured protocol and scope.

Study 3: does structure in memory damp noise-driven updating? In a stable routing world where 15% of feedback is misleadingly inverted, a wrongly punished correct action carries no information about the rule; in a changing world, the same surprise would be evidence of change. People adjust their learning to this distinction (Behrens et al., 2007; Nassar et al., 2010). The registered predictions test whether structured memory helps agents make the same distinction.

Result: all three predictions were contradicted. The short-versus-long memory ranking does not reverse between the noisy and the changing regime. The long transcript is less overconfident than the short one, not more. And the world-model substrate, built to revise itself the moment one of its predictions fails, abandoned still-correct behavior far more often than free-form notes rather than less. The run records show why: under lying feedback, every misleading rejection falsifies an explicit prediction, and every falsification triggers an immediate rewrite. The substrate we designed to make revision principled made it constant instead.

Evidence: Figure 6 shows the over-update rates. No measurable agent pair reverses sign between regimes. The overconfidence delta between the 120-step and 5-step transcripts is -0.084 $[-0.121, -0.047]$, on the

opposite side of zero from the prediction. The world model over-updates at **0.81** against notes’ **0.30**, a paired difference of $[0.35, 0.70]$. The capacity gradient survives: the 5-step transcript over-updates at **0.96** and loses -0.333 accuracy against fast+slow $([-0.458, -0.208])$. The notes baseline is similar across the three models (0.19 for deepseek-v4.1-flash, 0.28 for gpt-5.6-luna, 0.37 for qwen3.8-27b), all below the world model’s 0.81. The substrate arms run on a single model, so the world-model contradiction is scoped to gpt-5.6-luna.

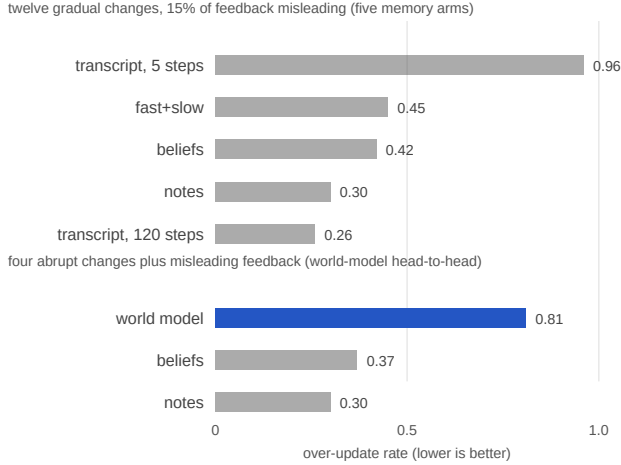


Figure 6: Over-update rate by memory substrate: the fraction of misleading rejections after which the agent abandoned a still-correct behavior at its next encounter with the same task; lower is better. The world model (blue) was registered to have the lowest rate of its cohort and instead has the highest: each lie falsifies one of its explicit predictions, and each falsification triggers an immediate rewrite.

Study 4: late feedback in a moving world.

Outcomes arrive 0, 1, 3, or 6 steps late, in a stable world and in a changing one. The intuition behind the registered prediction was that delay is a nuisance when nothing changes and expensive when something does, because stale credit lands on beliefs that have already moved on.

Result: a six-step delay is expensive for every model, and not only under change. The changing-world effect is supported for qwen3.8-27b and directionally consistent for the other two; the stable-world control shows losses just as large, with two separating from zero. These agents therefore handle delayed credit badly everywhere, and the available paired comparisons do not isolate a change-specific cost. A related registered prediction, that changes paced to wait for recovery beat

a fixed timer, leans positive on all three models without resolving.

Evidence: Figure 7 shows the paired effect of a six-step delay in both worlds. Changing world: qwen3.8-27b -0.417 $[-0.667, -0.167]$ (verdict: supported, scoped to that agent), deepseek-v4.1-flash -0.389 $[-0.667, +0.083]$, gpt-5.6-luna -0.250 $[-0.667, +0.083]$. Stable world: qwen3.8-27b -0.556 $[-0.667, -0.500]$, deepseek-v4.1-flash -0.444 $[-0.583, -0.333]$, gpt-5.6-luna -0.278 $[-0.667, +0.167]$. Adaptive versus fixed pacing: $+0.067, +0.067, +0.022$, every interval touching zero.

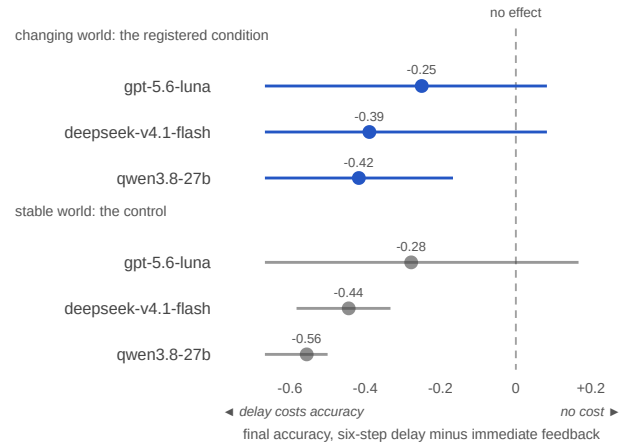


Figure 7: What a six-step feedback delay costs, per model and world: paired difference in final accuracy, delayed minus immediate. The registered prediction concerned the changing world (blue); the stable-world control (grey) shows the cost is at least as large when nothing changes, so the change-specific part of the claim remains open.

6 Limitations and future work

Realism and measurability. DriftLab’s instrumentation requires owning the world’s mechanics. Recorded per-step truth, byte-identical seed pairing, and drift as a controlled variable all depend on it, and naturally occurring artifacts do not allow it. The worlds are therefore not realistic in the ecological sense, however much realistic structure they carry (prose records, an evolving internal library, retail-shaped demand), and every result inherits that. What a synthetic world cannot supply is the unknown structure of real artifacts, the parts nobody thought to invent. We view realistic-artifact benchmarks such as EvoArena (Xu et al., 2026) as complements rather than competitors: instrumented cores measure the dynamics precisely, ecological benchmarks check that the

phenomena appear in the wild, and disagreement between the two is itself information.

Agent population. Every number in this paper comes from three models: gpt-5.6-luna and two open-weights models, deepseek-v4.1-flash and qwen3.8-27b. They were chosen for the best intelligence-to-cost profile available to us rather than for peak capability, so the results say nothing yet about frontier models. A second confound is structural: the model that plays the episodes is also the model that consolidates each memory substrate, so a substrate comparison is really a model-substrate interaction, and a substrate that fails on one model could succeed on a model that writes better notes. Verdicts are scope-qualified for exactly this reason, and the scope is narrow.

Data coverage and score completeness. The artifact is a completed analysis of the runs available in the repository, not a claim that every registered protocol has a full factorial result. The three-model coverage is broad for the shared notes profile, but substrate ablations are concentrated on gpt-5.6-luna: the open models were not tested across the full none/transcript/notes/skills/beliefs/world-model family. The form world has a completed Luna substrate comparison but no equivalent three-model comparison; inventory has three-model notes runs but no matched scheduled-versus-self-caused pairs; claims has complete Luna and DeepSeek paired arms but only a smaller Qwen notes arm; and campaign has three-model notes characterization but no matched notes-versus-no-memory comparison. H003 has only two matched endogenous-versus-scheduled pairs in the available extension, H005 has at most one confidence-lead run per relevant model, and H007 has zero matched wiped/unwiped pairs. H008 has matched adversarial comparisons, but none separates from zero. H001 and H004 are therefore partial families rather than broad confirmations; H002 is supported only for the observed Luna transcript/form cell; H006 has several contradictions but its real-change world-model comparison has only two matched pairs. Some reported scores are likewise partial: when a world does not expose the component required for knowledge, epistemics, or adaptation, the aggregate is incomplete or omitted from the compact characterization table. Missing cells are reported as missing, and no interpretation relies on treating an interrupted or unmeasured arm as a null result.

Future work. Three directions follow directly. First, widening the population: frontier models, and production agent systems evaluated as shipped rather than re-implemented. Second, running the open-weights models on owned hardware rather than through an API, which opens the way for studying adaptation at the weight level. The same instrument that compares in-context memory substrates can compare a per-episode adapter or a continually finetuned model against them, with detection latency, retention, and over-update measured identically. Third, further pushing the realism of the test environments within the established design constraints: full multi-file repositories with cross-file tickets, multi-session horizons with context severed between sessions, and drift schedules replayed from real versioned artifacts such as library changelogs, so a synthetic finding can be checked against a naturally occurring trace of the same shape.

7 Conclusion

Static benchmarks measure how competent an agent is on a fixed task. DriftLab measures what happens to that competence when the task changes: slowly, cosmetically, because of the agent, or in opposition to it. The design assumes that (a) hidden truth about the world is available at test time, (b) drift is a first-class variable, (c) capability is measured in the context of how an agent deals with latent change from successive encounters. Together these assumptions make adaptation a measurable, comparable, and transparently reported quantity. The testbed, all twenty protocols, and full run records are designed for release with the characterization results.

References

- C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR 2024*. [arXiv:2310.06770](#).
- G. Mialon, C. Fourier, C. Swift, T. Wolf, Y. LeCun, T. Scialom. GAIA: A Benchmark for General AI Assistants. [arXiv:2311.12983](#), 2023.
- S. Zhou et al. WebArena: A Realistic Web Environment for Building Autonomous Agents. [arXiv:2307.13854](#), 2023.
- X. Liu et al. AgentBench: Evaluating LLMs as Agents. *ICLR 2024*. [arXiv:2308.03688](#).
- S. Yao, N. Shinn, P. Razavi, K. Narasimhan. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World

- Domains. [arXiv:2406.12045](#), 2024.
- C.-K. Wu et al. StreamBench: Towards Benchmarking Continuous Improvement of Language Agents. *NeurIPS 2024 Datasets & Benchmarks*. [arXiv:2406.08747](#).
- J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, A. Bouchachia. A Survey on Concept Drift Adaptation. *ACM Computing Surveys* 46(4), 2014. [doi:10.1145/2523813](#)
- K. Khetarpal, M. Riemer, I. Rish, D. Precup. Towards Continual Reinforcement Learning: A Review and Perspectives. *JAIR* 75:1401-1476, 2022. [arXiv:2012.13490](#)
- M. McCloskey, N. J. Cohen. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. *Psychology of Learning and Motivation* 24, 1989. [doi:10.1016/S0079-7421\(08\)60536-8](#)
- A. Garivier, E. Moulines. On Upper-Confidence Bound Policies for Switching Bandit Problems. *ALT 2011*. [arXiv:0805.3415](#)
- N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS 2023*. [arXiv:2303.11366](#).
- G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. [arXiv:2305.16291](#), 2023.
- C. Packer et al. MemGPT: Towards LLMs as Operating Systems. [arXiv:2310.08560](#), 2023.
- J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. *UIST 2023*. [arXiv:2304.03442](#).
- M. Xiong et al. Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs. *ICLR 2024*. [arXiv:2306.13063](#).
- K. Tian et al. Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback. *EMNLP 2023*. [arXiv:2305.14975](#).
- T. E. J. Behrens, M. W. Woolrich, M. E. Walton, M. F. S. Rushworth. Learning the Value of Information in an Uncertain World. *Nature Neuroscience* 10:1214-1221, 2007. [doi:10.1038/nn1954](#)
- M. R. Nassar, R. C. Wilson, B. Heasley, J. I. Gold. An Approximately Bayesian Delta-Rule Model Explains the Dynamics of Belief Updating in a Changing Environment. *Journal of Neuroscience* 30(37):12366-12378, 2010. [doi:10.1523/JNEUROSCI.0822-10.2010](#)
- J. Xu, Q. Li, J. Wu, Y. Lan, et al. EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments. [arXiv:2606.13681](#), 2026.
- D. Wu et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *ICLR 2025*. [arXiv:2410.10813](#).
- M. Chu, X. B. Zhang, K. Q. Lin, et al. Agentic World Modeling: Foundations, Capabilities, Laws, and Beyond. [arXiv:2604.22748](#), 2026.
- K. Iten, B. Lee, C. Li, et al. Model-Based Reinforcement Learning for Control under Time-Varying Dynamics. [arXiv:2604.02260](#), 2026.
- D. Lopez-Paz, M. Ranzato. Gradient Episodic Memory for Continual Learning. *NeurIPS 2017*. [arXiv:1706.08840](#).
- J. Kirkpatrick et al. Overcoming Catastrophic Forgetting in Neural Networks. *PNAS* 114(13), 2017. [arXiv:1612.00796](#).
- I. Cabrera Martin, S. Mukherjee, A. Baimagambetov, J. Vanschoren. Evolving Machine Learning in Non-Stationary Environments: A Unified Survey of Drift, Forgetting, and Adaptation. [arXiv:2505.17902](#), 2025.

Appendix A: The six worlds

Each world is a job at a fictional company. The agent is never told it is in an experiment: episodes begin with a job description, changes arrive unannounced or as office memos, and probes are phrased as a supervisor’s question. Observations carry realistic, signal-free surface detail (names, ticket numbers, narrative text) drawn deterministically from (seed, step).

World	Drift type	Meaning in the world
<code>rule_world</code>	substantive	A routing rule flips, abruptly or migrating key by key.
<code>rule_world</code>	cosmetic	The request is re-rendered in a different format and field order.
<code>rule_world</code>	endogenous	An overloaded desk sheds a request type.
<code>form_filler</code>	substantive	A field is renamed, or a format, requirement, or cap changes.
<code>form_filler</code>	cosmetic	The intake note is restructured as prose, JSON, key-value lines, or TSV.
<code>form_filler</code>	endogenous	IT starts accepting a wrong field name the agent keeps typing.
<code>inventory</code>	substantive	The demand rate jumps or follows a random walk.
<code>inventory</code>	cosmetic	The daily report is re-worded.
<code>inventory</code>	endogenous	A large order strains the supplier and slows deliveries.
<code>codebase</code>	substantive	The style guide is revised or a library helper is deprecated.
<code>codebase</code>	cosmetic	The ticket template is re-worded.
<code>codebase</code>	endogenous	The reviewer adopts habits from the agent’s submissions.
<code>claims_desk</code>	substantive	The fraud signature moves abruptly or through a gradual handover.
<code>claims_desk</code>	cosmetic	The claim narrative template is re-worded.
<code>claims_desk</code>	endogenous/adversarial	The ring mimics the traffic the agent approves.
<code>campaign_desk</code>	substantive	Engagement rates are redrawn in an audience shift.
<code>campaign_desk</code>	cosmetic	Angle descriptions are re-worded.
<code>campaign_desk</code>	endogenous	An angle used relentlessly wears out and recovers with rest.

Worlds declare capabilities (substantive and cosmetic change, novelty, probes, task keys, confidence elicitation, endogenous mechanisms, consultation); protocols declare what they need, and a mismatch refuses at startup. The **World** column identifies the environment, while **Row type** identifies whether the row describes the job itself or a drift mechanism. “Endogenous/adversarial” marks the claims desk’s special case: its endogenous change is actively aimed at the agent.

Table 2: The six worlds and their representative drift mechanisms. Each row is explicitly labeled by its world and drift type. Substantive change moves the ground truth; cosmetic change moves only appearances; endogenous change is triggered by the agent’s own behavior, and in `claims_desk` it is adversarial (aimed at the agent). Gradual and abrupt variants of substantive change exist where noted in [Section 3.2](#).

Routing (`rule_world`). Requests arrive one at a time as short intake emails (a ticket number, a named sender, a subject line, a note from the requester), each carrying a type, region, and size. A hidden rule table with nested exceptions (a base rule per type, regional exceptions, size exceptions to those) maps every combination to the one desk that will accept it; the agent picks a desk and hears accepted or rejected, nothing more. Because the same combinations keep recurring, the run record can count exactly how many encounters an agent needs to notice and adapt.

Formally: non-stationary hierarchical classification over a hidden decision table.

Hooks, flags, and knobs: `change_latent` (a rule flips; with `transition_window` set, it migrates key by key), `change_surface` (`surface_strength` selects rewording or full structural re-rendering), `introduce_novelty`, `probe`, `consult`, and the endogenous flag (an overloaded desk sheds a request type). Knobs: `depth` and `n_types` size the rule table, `feedback_noise` inverts a share of feedback, `warmup` delays the first change.

Job description (the system prompt, verbatim):

```
text You work on the intake desk at Meridian Logistics. Requests arrive one at a time; for each one, assign it to the handling option you believe is right. The back office tells you whether the assignment was accepted. Nobody has written the routing procedures down, so you will have to work them out on the job. End every reply with a line of the form: CHOICE: <A|B|C|D>
```

A rendered observation (seed 0, step 0):

```
text Ticket REQ-2400 · from Yuki Rossi (Partner Accounts) Subject: transfer request - south region "Our client has been asking about this twice already." We have received a large transfer originating from the south region and need it assigned to the right desk. Options: A (standard queue), B (priority desk), C (regional office), D (external partner) Which option do you choose?
```

Forms (form_filler). Each step presents an intake note, a typed-up phone order or a voicemail transcription in which every field value appears verbatim and quoted, to be submitted as a filled form matching a hidden schema of field names, formats, requirements, and caps. Submissions are checked field by field, with a few attempts allowed per record; richer feedback levels name what was wrong. A substantive change mutates the schema itself; a cosmetic change restructures the note (prose, JSON, key-value lines, TSV) while the form stays the same. Formally: structured generation under schema drift.

Hooks, flags, and knobs: `change_latent` (the schema mutates: a rename, a format, a requirement, or a cap), `change_surface` (`surface_strength` restructures the intake note across prose, JSON, key-value, and TSV

renderings; `record_style` sets the base rendering), `introduce_novelty` (a new field), `probe`, and the endogenous flag (IT aliases a wrong field name the agent keeps typing). Knobs: `max_attempts` per record, `feedback` richness.

Job description (the system prompt, verbatim):

```
text You are a data-entry specialist at Meridian Logistics. You receive order records and must enter each one into the company's order form. The system rejects invalid submissions and reports errors; the form has no manual. Reply with the completed form as a single JSON object on its own line, prefixed by FORM: FORM: {"field": "value", ...}
```

A rendered observation (seed 0, step 0):

```
```text Intake batch ORD-5100, forwarded by Yuki Rossi at the front desk. "Original was handwritten; transcribed by reception." Order form (version 1). Fields: cust, email, order_date, priority, qty, ship_country. Attempt 1 of 3.
```

Source record: Transcribed from the customer's voicemail: the customer is "Eli Reyes"; email given as "user559@example.com"; quantity given as "33"; shipping\_country given as "Germany"; the order\_date is "2026-01-03"; priority\_level given as "0". ````

**Inventory (inventory).** Each morning the agent sees stock on hand, orders in transit, and yesterday's sales (capped by what was in stock), then orders a quantity that arrives after a lead time. Demand is drawn from a hidden rate; under the retail regime that rate is shaped like a real sales series, a weekday cycle over the base level with occasional promotion spikes, and a substantive change jumps the level while preserving the weekly shape. The day's score combines holding costs, lost sales, and an ordering fee. The right order quantity is a calculation, not a choice from a list, and the world's rewards are costs, so accuracy-style metrics stay unmeasured here by design. Formally: non-stationary inventory control under censored Poisson demand.

*Hooks, flags, and knobs:* `change_latent` (the demand level jumps; under the retail regime the weekly shape is preserved), `change_surface` (the daily report is reworded), `probe` (a hypothetical from the manager), and the endogenous flag (an order above `big_order` strains the supplier for `strain_days`). Knobs: `regime` selects

the demand generator (stationary, seasonal, drifting, retail), `lead_time`, and the three cost rates.

*Job description (the system prompt, verbatim):*

```
text You are the buyer for one product line at Meridian Retail. Each morning you see stock on hand, orders in transit and yesterday's sales, and place today's order (it arrives in 2 days unless the report says otherwise). Your job is to keep total cost down. Costs: 0.1 per unit held per day, 1.0 per unit of demand you could not serve, 0.5 fixed cost per non-zero order. End your reply with a line: ORDER: <integer>
```

*A rendered observation (seed 0, step 0):*

```
text Day 1, Monday. Stock on hand: 18 units. Arriving over the next 2 days: [0, 0]. Yesterday's sales: n/a. Sales over the last 14 days: none yet. Ops note: Cycle count scheduled for the end of the month. How many units do you order today?
```

**Code (codebase).** Tickets ask for small Python functions, a third of which build on the team's internal utility library (identifier, date, text, and money helpers provided in scope). Every submission is executed in a sandbox against the ticket's tests and also checked against a style guide the agent can never read; full credit needs both, and reviews can run several rounds per ticket. A substantive change revises the style guide, so code that used to pass review stops passing, or deprecates a library helper: the old name keeps working but now fails review, and the onboarding description silently goes stale, so the deprecation has to be discovered from review feedback rather than announced. Formally: constrained code synthesis with a fixed test oracle and drifting conventions.

*Hooks, flags, and knobs:* `change_latent` (the style guide is revised; with `api_drift` set, it may instead deprecate a library helper), `change_surface` (the ticket template is re-worded), `consult` (the reviewer names one live rule), and the endogenous flag (the reviewer adopts habits from the agent's submissions). Knobs: `session_length`, `review_rounds`, `feedback` richness.

*Job description (the system prompt, verbatim):*

```
text You are a developer on the platform team at Meridian. Tickets ask you to implement small
```

```
functions against the team's internal library; CI runs the tests and a reviewer checks the code against the team's style guide, which is not written down anywhere you can see. A helper `log(msg)` is available in scope, along with the team utilities (as of your onboarding): fmt_id(x) - display reference for a numeric id, e.g. 7 -> 'MD-000007'; parse_dt(s) - split an ISO date string into an (y, m, d) tuple of ints; clean(s) - collapse whitespace and lowercase; to_cents(amount) - convert a currency amount to integer cents. Reply with the complete function source inside one code block: python
```

*A rendered observation (seed 0, step 0):*

```
text (reported by Yuki Rossi · P3 · "Blocks a downstream cleanup ticket.") Ticket #1000: implement `order_year(s)`. Return the year of an ISO order date string, using the team date helper.
```

**Fraud (claims\_desk).** Claims arrive as short narratives, drawn from a pool of incident stories and carrying a signal-free adjuster's note, with three underlying attributes (type, filing channel, amount band). A fraud ring operates with a signature, a pair of attribute values its claims always carry, and legitimate claims never fully match it, so the task reduces to tracking the signature as it moves. With the adversary active, the ring watches what gets through and re-styles its claims to mimic the traffic the agent has been approving. Formally: adversarial concept drift with a best-responding drift generator.

*Hooks, flags, and knobs:* `change_latent` (the fraud signature moves; with `transition_window` set, old and new playbooks run side by side during the handover), `change_surface` (narrative templates rotate), `consult` (the audit desk reveals one claim's truth), and the endogenous flag, which here is the adversary: the ring re-styles its claims against what the agent approves, reconsidering every `adapt_every` claims. Knobs: `fraud_share`, `n_types`, `warmup`.

*Job description (the system prompt, verbatim):*

```
text You review insurance claims at Meridian Assurance. Claims arrive one at a time; approve the legitimate ones and reject the fraudulent ones. Fraud exists and fraud tactics change over
```

time; you are never told the current pattern. End your reply with a line: DECISION: APPROVE or DECISION: REJECT

*A rendered observation (seed 0, step 0):*

text Claim CLM-7001 - claimant Kofi Kim, policy MA-34280, auto cover. Windscreen cracked by road debris on Mar 3. Filing: submitted through the customer portal (online). Requested payout: \$826 (small band). Adjuster note: Documents scanned and attached to the file. Category: auto . Channel: online . Amount: small Do you approve or reject this claim?

**Campaigns (campaign\_desk).** Each day (a named weekday: audience activity follows a weekly cycle that scales all angles equally, so report totals gain realistic texture while the best angle is unchanged) the agent picks one of four outreach angles; every push lands or falls flat according to hidden per-angle engagement rates, but no single result is ever shown. The only feedback is a pooled total every reporting period, with no breakdown, so connecting reports back to one’s own recorded choices is the task. With fatigue active, an angle used relentlessly wears out and recovers with rest. Formally: a non-stationary bandit under batched, delayed, aggregate feedback.

*Hooks, flags, and knobs:* change\_latent (engagement rates are redrawn), change\_surface (angle descriptions re-worded), and the endogenous flag (fatigue: a relentlessly used angle wears out and recovers with rest). Knobs: report\_every sets the pooled-report cadence, regime adds a slow random walk, weekly the weekday activity cycle.

*Job description (the system prompt, verbatim):*

text You run the outreach campaign at Meridian Media. Each day you pick the angle for that day’s push. You never see how a single push performs: results arrive only in a pooled report every week or so, with no per-day breakdown. Audience tastes change over time. End your reply with a line: ANGLE: <letter>

*A rendered observation (seed 0, step 0):*

text Day 1, Monday. Morning stand-up note: Legal signed off on all four running angles. Pick today’s angle: A) a price cut B) a quality story

C) limited-time urgency D) a customer testimonial Which angle do you push today?

**Table 3:** The drift vectors and instruments each world exposes (columns abbreviate rule\_world, form\_filler, inventory, codebase, claims\_desk, campaign\_desk; an x means exposed, an empty cell means not). Abrupt and gradual are substantive change; gradual takes a world-specific form (key-by-key migration in routing, a drifting rate in inventory, a playbook handover in claims, a random walk in campaigns). Cosmetic change is structural re-rendering in the routing and form worlds and rewording elsewhere. Endogenous change is triggered by the agent’s own behavior; adversarial change is aimed by an opponent that observes outcomes. Novelty introduces never-seen task types; probes are unscored diagnostic questions; consultation lets the agent buy a true answer at the price of a step; task keys identify recurring tasks, which is what makes encounter-based latencies measurable. Every world elicits confidence when a protocol asks for it. A protocol that needs a vector a world does not expose refuses at startup.

Vector	rule	form	inv	code	claims	camp
abrupt	x	x	x	x	x	x
substantive						
gradual	x		x		x	x
substantive						
cosmetic	x	x	x	x	x	x
endogenous	x	x	x	x	x	x
adversarial					x	
novelty	x	x				
probes	x	x	x			
consultation	x			x	x	
task keys	x				x	

With consultation enabled, a world prices information itself: the agent may spend a step asking (the floor manager names the right desk, the audit desk reveals whether the claim is fraud, the reviewer names one live style rule) and forfeits the step’s reward.

## Appendix B: The twenty experimental protocols

Each experiment is a versioned schedule of drift, notices, and probes, named for the question it asks. The registry entry for each contains the design, concrete parameters, regimes, and any registered predictions; predictions currently cover experiments 2, 4, 5, 6, 9, 13, 15, 16, 17, 18, 19, and 20.

Every experiment configures its world the same way (Figure 2). A shared set of defaults first neutralizes the worlds’ internal timers, so nothing changes unless the protocol schedules it: the form world’s version clock and the code world’s session clock are effectively off, the routing world plays 120 steps over a depth-2 rule table, and inventory demand carries its retail texture (weekday cycle and promotion spikes) over a flat base level, so the level moves only on a scheduled jump. The code world’s library deprecations are enabled, so scheduled substantive changes there mix style-guide revisions with helper deprecations. On top of the defaults, the experiment sets the knobs its question needs (structural rather than label-only cosmetic change, an endogenous or adversarial flag, a transition window for gradual drift) and fires every change through its per-step schedule, so the run record logs the exact step and key set of each change event. Reduced modes shrink both length and task space together, keeping the encounter density that latency measurement needs.

**01: Which task-selection strategy trains an agent fastest?** A scripted teacher controls which task the agent trains on next. Five strategies compete: picking at random, easiest first, aiming for tasks the agent currently gets right about half the time, replaying recent failures, and targeting that frontier with a bonus for task types the agent has never seen. Every 25 steps the agent is scored on the same held-out probe set, so the strategies are compared on what the agent actually learned rather than on what it practiced. Episodes run 150 steps on the routing world. *Configuration:* Routing world at rule depth 3 with a raised exception rate; no scheduled drift, the teacher’s task selection is the only manipulated input; a notes agent learns; probes are a fixed held-out set.

**02: Does detecting a rule change predict recovering from it?** Six rule changes arrive at even

intervals after a 20-step warmup, and agents that differ only in their memory substrate play the same episodes. Detection, recovery, and stability latencies are measured separately, because an agent can stop trusting its old answer well before it finds the new one, and the gap between those two moments turns out to depend on how it remembers. 120 steps. *Configuration:* Canonically the routing world (any world with the substantive-change hook qualifies); six latent changes spaced evenly after a 20-step warmup; arms are the tabular floor plus none, transcript, notes, and skills memories.

**03: Can an agent tell a changed rule from one it never knew?** Rule changes and never-seen task types are mixed through the same episode, so a rejection is genuinely ambiguous. After every rejection the agent is asked, off the record, which of the two it believes happened. The diagnosis is scored against the recorded truth, and so is the attempt that follows it, since a correct diagnosis should produce a better next move. 130 steps. *Configuration:* Routing world sized to five request types; scheduled latent changes interleaved with novelty introductions; the attribution probe fires after every rejection; transcript and notes arms.

**04: Do agents over-react to purely cosmetic change?** At three points in the episode something happens: nothing, a cosmetic change, a substantive change, or both, depending on the regime. The accuracy dip after each kind of event gives an over-reaction index. In v2 the cosmetic change is structural, re-rendering the task in a different format with a different field order, because v1’s synonym-level rewording produced no dip in any frontier model we tried. 120 steps. *Configuration:* Canonically the form world (any world with both change hooks qualifies); `surface_strength` set to structural; events at steps 30, 60, and 90 fire the latent hook, the surface hook, both, or neither according to the regime; transcript and notes arms.

**05: Which memory window maximizes accuracy under drift?** Six retention settings, transcript windows from 5 to 120 steps plus note budgets, are crossed with three drift rates: no changes, four, or twelve. In v2 the tasks come from a recurring caseload, so even a 5-step window genuinely meets the same task again; in v1 the full task space starved the short windows, an audit finding that forced the version bump. 120 steps.

*Configuration:* Routing world drawing tasks from a recurring caseload; zero, four, or twelve latent changes spaced after warmup; arms are transcript windows of 5, 15, 40, and 120 steps plus budgeted notes.

**06: Does stated confidence drop before behavior recovers?** The agent states a confidence from 0 to 100 with every action while six changes land. The run is scored on calibration, on overconfidence, and on the timing between two events: the moment stated confidence sags on affected tasks and the moment behavior recovers. The registered sign claim is that confidence sags first, which would make it usable as a change detector. 120 steps. *Configuration:* Routing world with `ask_confidence` on, so every action carries a stated confidence; six spaced latent changes; transcript and notes arms.

**07: Can agents exploit a predictable change schedule?** The same number of changes arrives on three different clocks: irregular, exactly periodic, or triggered by the agent’s own performance. If timing is learnable at all, the dips after later changes should shrink on the periodic clock and stay flat on the irregular one. 160 steps. *Configuration:* Routing world; the same change count delivered by three clocks: evenly spaced, spaced with a random jitter of a few steps, or triggered by the agent’s own performance; notes agent.

**08: Does explanatory feedback beat outcome-only feedback?** Three worlds (forms, inventory, code) each run at three levels of feedback richness, from a bare outcome to a full explanation of what was wrong. This is the one protocol that keeps the worlds’ internal clocks running: the form schema mutates every 15 steps, inventory demand changes season every 30 days, and the code world revises its style guide between 8-ticket sessions. The steps right after each change are scored separately, because an explanation should be worth the most exactly when the rules have just moved. *Configuration:* The exception that keeps world clocks on: form world with a schema version every 15 steps, inventory in the seasonal regime with a change every 30 days, code world with 8-ticket sessions; the regime sets each world’s `feedback` level; notes agent consolidating every 8 steps.

**09: How much accuracy does delayed feedback cost under drift?** Outcomes arrive 0, 1, 3, or 6 steps late, in a stable world and in a changing one. Delay should be a minor nuisance when nothing changes and

expensive when something does, because stale credit lands on beliefs that have already moved on; the registered prediction is that delay costs accuracy under change. 120 steps. *Configuration:* Routing world behind a delay wrapper that withholds each outcome for 0, 1, 3, or 6 steps; crossed with a stable world and a changing one (six spaced changes); transcript and notes arms.

**10: Do minimal contrast pairs speed up rule learning?** The teacher presents minimal pairs: two tasks that differ in exactly one attribute and have different correct answers, shown back to back. The contrast ordering competes against random and frontier ordering, with learning scored on held-out probes rather than the training stream. 150 steps. *Configuration:* Routing world at depth 3 with a raised exception rate; the teacher orders the stream as contrast pairs, at random, or by frontier targeting; notes agent; held-out probes.

**11: Do advance warnings help, and what do false alarms cost?** Office memos announce upcoming changes 0, 1, or 3 steps in advance. One regime makes half the memos false alarms, and the measurement of interest is the damage a false alarm does: the dip it causes on tasks that never changed at all. 120 steps. *Configuration:* Routing world; six spaced latent changes announced through the notice channel 0, 1, or 3 steps ahead; one regime replaces half the memos with false alarms about keys that never change; notes agent.

**12: Which note-consolidation trigger wins per dollar?** Consolidation schedules are compared at equal cost: rewrite the notes on a fixed interval, rewrite them after failures, do both, or never rewrite at all. The score is accuracy per dollar, because consolidation is itself an LLM call and rewriting after every failure is not free. 120 steps. *Configuration:* Routing world; six spaced latent changes; the arms are one notes agent per consolidation trigger (fixed interval, after failures, both, never), compared at matched spend.

**13: Does recovery-paced change beat a fixed schedule?** Six changes arrive on a fixed timer, adaptively (each one waits until the agent has recovered from the last), or adversarially (each one lands the moment recovery is detected). The registered prediction favors adaptive pacing over the timer. 150 steps. *Configuration:* Routing world; six latent changes paced by a fixed timer, adaptively (each waits for recovery), or

adversarially (each lands at detected recovery), driven by a per-step monitor; notes agent.

**14: Does learned knowledge transfer to fresh rules and novel tasks?** After 100 steps of training, three evaluation phases open with feedback-free probes: familiar tasks, fresh rules on familiar types, and entirely novel types. A fresh-agent control plays the same evaluation, so whatever the trained agent carries over from training is separated from what any agent could do cold. 160 steps. *Configuration:* Routing world at depth 3; 100 training steps, then probe-only evaluation phases with fresh rules and novel types; a freshly initialized control agent plays the same phases; notes agent.

**15: Are self-caused changes harder to recover from than scheduled ones?** In the endogenous regime nothing is scheduled: the world changes only in reaction to the agent’s own behavior. A stable control and a scheduled control matched in expected change count complete the comparison. The registered prediction is that self-caused changes are slower to recover from, because no external moment marks them. 150 steps. *Configuration:* Canonically the inventory world (any world with an endogenous mechanism qualifies); the endogenous regime arms the world’s self-caused mechanism with nothing scheduled, against a stable control and a scheduled control of four spaced changes; transcript and notes arms.

**16: Do agents over-update on misleading feedback?** Three regimes make the same surprising outcome mean different things: four abrupt changes, twelve gradual ones, or a stable world in which 15% of feedback is misleadingly inverted. Five memory substrates play all three regimes and state confidence throughout. The registered predictions cover a memory-ranking reversal between regimes and an overconfidence effect; [Section 5](#) reports both contradicted. 120 steps. *Configuration:* Routing world with task keys and confidence elicitation on; regimes are four abrupt changes, twelve gradual ones, or a stable world with 15% of feedback inverted; five memory arms (5- and 120-step transcripts, notes, beliefs, fast+slow).

**17: What does consolidation preserve across a context wipe?** Halfway through the episode, some agents lose their raw history: a transcript agent loses everything it had, a notes agent only what it had not yet consolidated. Wiped and unwiped versions of the

same memory play identical seeds, so the cost of the wipe is a clean paired difference, and the registered predictions price it. 120 steps. *Configuration:* Routing world; six spaced latent changes; wiped arms erase their raw history at midpoint through the substrate’s wipe hook while consolidated artifacts survive; transcript and notes, wiped and unwiped.

**18: Is recovery slower when an adversary aims the drift?** The fraud world runs under no shifts, timed shifts, and adversarial shifts, with the ring reconsidering its tactics every 20 claims and moving whenever at least half of its recent claims were caught. The registered prediction, that recovery from aimed shifts is slower, is the subject of Study 2 in [Section 5](#). 150 steps. *Configuration:* Claims world; regimes are no shifts, four scheduled shifts, and the armed adversary (the ring reconsiders every 20 claims and moves when at least half its recent claims were caught); transcript and notes arms.

**19: Can agents assign credit through pooled, delayed reports?** The campaign world never scores a single push; results arrive only as periodic totals, and the cadence varies from every 5 to every 20 pushes while two audience shifts land mid-run. The registered prediction is that under pooled feedback memory is a precondition rather than an optimization: an agent with no record of its own choices has nothing to connect a report to. 120 steps. *Configuration:* Campaign world at report cadences of 5, 10, and 20 pushes; two scheduled audience shifts mid-run; arms are none, transcript, and notes memories.

**20: Does prediction-gated revision reduce over-updating?** Notes, structured beliefs, and the self-revising world model play four abrupt changes and a lying-feedback regime. The registered predictions state that the world model over-updates less than notes under noise and recovers no slower under real change; [Section 5](#) reports the first contradicted and the second inconclusive. 120 steps. *Configuration:* Routing world with lying feedback available; regimes are four abrupt changes and the noisy-stable condition; arms are notes, beliefs, and the world model.

## Appendix C: Agents and memory substrates

**Reference substrates.** All reference agents share one model and one loop and differ only in what persists between steps. The loop is deliberately simple: the world’s job description is the system prompt, and on every step the substrate’s recall text is prepended to the observation, so memory reaches the model as plain text in the user prompt and nothing else about the request changes between substrates. After each outcome the substrate records one experience line of the form `[step t] observation | did: action | outcome (+0.00): feedback`, and consolidating substrates buffer these lines and rewrite their artifact with one LLM call when their trigger fires: every 10 steps by default, immediately after a failure, both, or never, optionally under a character budget. Every rewrite is versioned into the run record, so the evolution of an agent’s memory is replayable after the fact.

The seven substrates, with the exact consolidation instructions where one exists:

**none** persists nothing; every step is played from the observation alone.

**transcript** keeps a sliding window of raw experience lines (40 steps by default; exp05 varies the window from 5 to 120), injected verbatim under “Recent history:”. Nothing is ever summarized.

**notes** maintains a free-form working file, injected as “Your notes:”. Its consolidation instruction: *“You maintain a concise working-notes file for an agent operating in an environment. Rewrite the notes to incorporate the new experiences. Keep what is useful for future decisions; drop or correct anything the evidence now contradicts. Plain text, under 300 words.”*

**skills** maintains a procedure library, injected as “Your procedures:”. Its instruction: *“You maintain a library of reusable procedures for an agent operating in an environment. Each procedure is one line: **WHEN** <conditions> **DO** <action>. Update the library from the new experiences: add procedures the evidence supports, and fix or delete procedures the evidence contradicts. Output only the procedure lines.”*

**beliefs** maintains declarative statements with revision conditions, injected as “Your current beliefs:”. Its instruction: *“You maintain a belief file for an agent*

*operating in an environment. Each line is one belief: **BELIEF:** <what you currently believe> | **CONFIDENCE:** <low/medium/high> | **WOULD CHANGE IF:** <evidence>. Update the file from the new experiences: raise or lower confidence with the evidence, rewrite beliefs the evidence contradicts, and delete beliefs that no longer apply. Output only belief lines.”*

**worldmodel** maintains explicit rule hypotheses and is the only substrate that changes the agent’s reply format: its recall appends a request to end every reply with **EXPECT:** <success|failure> and a short reason. A prediction that the next outcome contradicts is marked in the experience buffer as **PREDICTION WRONG** and triggers consolidation immediately rather than at the next interval; the made and wrong prediction counts feed the wrong-prediction metric of [Section 3.3](#). Its instruction: *“You maintain an explicit world model for an agent operating in an environment. Each line is one rule hypothesis: **RULE:** <scope / when it applies> -> <what holds or what to do> | **CONFIDENCE:** <low/medium/high> | **SINCE:** step <n> | **INVALID IF:** <evidence that would retire it>. Update the model from the new experiences: raise or lower confidence with the evidence, revise rules the evidence contradicts (reset their **SINCE** to the step of the contradiction), and delete rules that no longer apply. When outcomes contradict a high-confidence rule, prefer ‘the environment changed’ over ‘the rule was always wrong’. Output only **RULE** lines.”*

**fast+slow** composes a 10-step raw transcript with a notes file consolidated every 10 steps; the prompt carries both, notes first. Only the notes are exported across sessions.

The format instructions are prompts, not grammars: nothing validates the artifact, and whatever text the consolidation call produces is what future steps see. Transcript and notes support a mid-run context wipe in which the raw window and any unconsolidated buffer are erased and only the written artifact survives, isolating what consolidation is for (exp17).

**Robustness.** Timeouts and unparseable replies become the world’s seeded default action and count as parse failures; transient provider errors (rate limits, dropped connections, upstream 5xx) are retried with bounded backoff before an episode is failed.

**Fault isolation and resumption.** Every episode has a deterministic identity (agent, regime, seed) and is written atomically on completion, so any grid resumes

at episode granularity and later runs extend earlier ones instead of repeating them. One episode’s hard failure never discards its siblings’ finished work.